

О проекте

Задача. Проект SQL-HR решает прикладную задачу поиска и первичного отбора кандидатов по внутренней базе заказчика (резюме/анкеты/карточки кандидатов) в ответ на запрос HR/менеджера на естественном языке.

Специфика задачи:

- Чувствительные данные и ограничения безопасности: данные не могут покидать контур заказчика, поэтому вся обработка выполняется on-prem.
- Ограниченные вычислительные ресурсы: инференс LLM может выполняться на CPU, что накладывает ограничения на количество итераций, длину контекста и допустимую задержку.

Механика работы (end-to-end).

1. Пользователь формулирует запрос в чате (например, роль + навыки + ограничения).
2. Диалоговый агент уточняет требования до достаточной формализации (что обязательно/желательно/нельзя; какие числовые пороги важны).
3. LLM переводит требования в структурированную схему запроса (structured input): набор условий по полям + логика AND/OR/NOT + числовые фильтры.
4. Поисковый агент выполняет три вариации (лучше требований, в соответствии с требованиями, ниже требований) sql запроса и производит поиск в БД по этим условиям, извлекает пул кандидатов.
5. Дополнительно выполняется ранжирование (cross-encoder), после чего возвращается топ-k кандидатов и (при необходимости) краткое обоснование по совпавшим критериям.

Принципиальная идея. LLM переводит естественный язык в контролируемый и интерпретируемый запрос, а поиск делегируется БД и легким моделям ранжирования. Это дает баланс между качеством, скоростью, приватностью и объяснимостью.

1) Что мы сделали

Мы разработали on-prem систему поиска кандидатов по внутренней базе заказчика (чувствительные данные, запрет на облачные вычисления). Система реализована как мультиагентное решение из двух агентов:

- **Agent A (диалоговый):** общается с пользователем в чате, уточняет требования к кандидату, собирает недостающие параметры, контролирует полноту запроса.
- **Agent B (поисковый):** выполняет поиск кандидатов в базе по формализованным критериям и возвращает результаты.

Agent A вызывает Agent B через tool, который принимает **требования на естественном языке** и возвращает **найденных кандидатов**. Ключевая идея: LLM используется преимущественно для **интерпретации намерения** и **перевода требований в структурированный запрос**, а сам поиск и фильтрация выполняются классическими механизмами БД.

2) Как устроен поиск: перевод NL → structured input → SQL/поиск в БД

Поиск кандидатов строится на **структурированном представлении запроса**, где требования разделяются на типы:

2.1. Текстовые поля (опыт, образование, навыки и т.п.)

Для каждого текстового поля формируются три группы ключевых терминов:

- Обязательные (логика AND): слова/фразы, которые должны встречаться.
- Желательные (логика OR): повышают релевантность, но не обязательны.
- Запрещённые (логика NOT): исключают кандидатов при наличии этих терминов.

2.2. Числовые и категориальные признаки

Отдельно задаются фильтры по:

- возрасту, стажу, грейду/уровню и т.п.;
- локации, занятости, формату работы (если есть в БД);
- другим строгим ограничениям.

2.3. Роль LLM

LLM делает **structured output**: из диалога и NL-требований генерирует заполненные поля схемы (обязательные/желательные/запрещенные термины + числовые фильтры). Это повышает предсказуемость и управляемость поиска: запрос объясним, редактируем и аудлируем.

3) Эволюция реализации: три архитектурные версии

Версия 1. «Строгая последовательность» (pipeline с перефразированием)

Цель: повысить recall за счет разнообразия формулировок.

1. Agent A генерирует несколько **вариаций запроса** с акцентом на разные аспекты (синонимы роли, навыков, формулировок).
2. Для каждой вариации выполняется поиск кандидатов (например, батчами по 10).
3. Если результатов нет или мало - LLM предлагает **упростить требования** (ослабить фильтры/убрать редкие термины).
4. Затем LLM выполняет **ранжирование** найденных кандидатов и возвращает итоговый список.

Сильная сторона: устойчивость к «неудачной формулировке» пользователя.

Слабая сторона: много итераций → высокая задержка.

Версия 2. «Более автономный поисковый агент» (итеративный loop с тремя tool)

Цель: сделать поиск более «реактивным» и похожим на работу исследователя.

Agent B получил цикл принятия решений и три действия:

1. **Задать уточняющий вопрос** Agent A (если требований недостаточно или есть конфликт).
2. **Сделать запрос в БД** (применить текущие фильтры).
3. **Вернуть результаты** (когда найдено достаточно кандидатов).

Сильная сторона: гибкость и адаптивность, меньше «жёсткого сценария».

Слабая сторона: задержка всё ещё может быть высокой (много шагов + длинные тексты резюме в контексте).

Версия 3. «Оптимизация под CPU и ограниченный контекст» (архитектурный хак)

Контекст: заказчик потребовал снизить время поиска. LLM работала на CPU, и основная проблема была в том, что:

- резюме кандидатов длинные;
- небольшая модель хуже работает на большом контексте;
- итеративная оценка кандидатов LLM-ом становится узким местом по времени.

Решение: переработали архитектуру так, чтобы:

- уменьшить число итераций,
- сократить объем текста, который LLM должна «переваривать»,
- заменить часть оценки кандидатов алгоритмическими шагами.

Ключевой прием версии 3:

1. Agent A/Agent B **генерирует сразу три структурированных запроса**:
 - **Ideal**: требования «выше среднего» (строже, чем минимум).
 - **Fit**: минимально достаточные требования.
 - **Fallback**: чуть ниже требований (как HR-стратегия расширения воронки).
2. По каждому запросу быстро извлекается пул кандидатов из БД.
3. Возвращается **лучшая группа**, при необходимости подбираем до лимита (например, 5 кандидатов) за счет Fit/Fallback.
4. Финальная сортировка выполняется **кросс-энкодером**, который ранжирует пары (запрос, кандидат) точнее и стабильнее, чем малая LLM в длинном контексте.

Итог: снижаем latency, при этом качество падает умеренно, потому что точность «последнего шага» обеспечивает cross-encoder.

4) Вклад

1. **On-prem архитектура поиска кандидатов**, где LLM используется как «переводчик смысла» в структурированный запрос, а основная работа делается БД (объяснимость + контроль + приватность).
2. **Архитектурная оптимизация под CPU-инференс**: отказ от тяжелой LLM-оценки длинных резюме и переход к трём запросам + cross-encoder reranking.
3. **Шаблон поиска HR**: Ideal/Fit/Fallback как системный способ управлять компромиссом precision/recall при жестких ограничениях на время.

5) Научный интерес

- **Агентные системы в условиях ограниченных ресурсов**: как сохранить качество, когда LLM работает на CPU и не тянет длинный контекст.
- **Оптимизация задержки** в retrieval/agentic системах без GPU за счет архитектурных решений и классических алгоритмов.
- **LLM-as-compiler**: LLM не «читает все резюме и рассуждает», а компилирует требования в запрос → повышается объяснимость и удобство аудита, что критично в корпоративных сценариях и при чувствительных данных.

6) Идея экспериментального сравнения

С чем сравнивать (блок бейзлайнов)

1. **Ваши версии**: v1 vs v2 vs v3.
2. **Та же логика, но большая LLM на GPU** (верхняя граница качества/скорости).
3. **Классика без LLM**:
 - keyword search + логические фильтры (без трех запросов);
 - BM25/TF-IDF.

4. **Векторный поиск:**
 - embeddings retrieval (возможно + фильтры по числовым признакам).
5. **“Один LLM делает всё”:**
 - RAG/agent, который сам читает резюме и выбирает кандидатов (обычно дорого по времени и контексту).
6. **Аналоги с открытым кодом** (если реально воспроизводимы).

Метрики

- **Latency:** среднее/медиана/TTFT.
- **Качество подбора:**
 - Recall@k / Precision@k (если есть разметка релевантности),
 - nDCG@k (если релевантность градуированная),
 - доля «подходящих» по критериям (rule-based проверка) + экспертная/LLM-оценка.

Решения из [Аналоги](#), у которых есть исходный код

7) Датасеты и разметка

Источники данных:

- русскоязычные публичные наборы резюме/вакансий (пример: порталы вакансий/карьерные траектории);
- англоязычные датасеты резюме (Kaggle/HuggingFace);
- синтетические профили кандидатов (для контролируемых тестов).

Разметка:

- либо ручная (экспертная),
- либо полуавтоматическая: LLM/agent-judge оценивает соответствие кандидата требованиям, с последующей валидацией на подмножестве.

В качестве датасета можно использовать:

Датасеты на русском:

- 1) [Educational and Career Trajectories in Russia \(Jobs in Russia portal / “Работа в России”\)](#)
- 2) Прошлый датасет

На английском языке:

- 1) [Kaggle “Resume Dataset” \(примеры резюме с LiveCareer\)](#)
- 2) [HuggingFace resume-dataset](#)

Также можно использовать синтетические датасеты

Разметку можно сделать через LLM или использовать агента судью, который будет оценивать качество найденных кандидатов и их корректность по группам.